Platform architecture and automation contract

Runtime layers

1. Edge and web: TLS, rate limiting, bot protection, secure headers, request IDs.

2. Identity: OIDC/OAuth 2.0, phishing-resistant MFA, session revocation, recovery controls.

3. Application: Next.js App Router, server-side policy checks, validated route handlers.

4. Domain: tenant, client, authorization, case, transcript, refund, notice, document, task, transmission, acknowledgment, payment, training, and audit modules.

5. Data: PostgreSQL-compatible transactional store, encrypted object vault, append-only audit events, queue/outbox, cache.

6. Integration: IRS-approved interfaces only, Stripe, email/SMS, storage, identity, and signed webhooks.

7. Delivery: GitHub Actions, dependency review, tests, preview, approval, production promotion, monitoring, and rollback.

RBAC and separation of duties

Policy is deny-by-default, tenant-scoped, MFA-gated, and server-enforced. UI visibility is never authorization. A preparer cannot approve the same transmission. High-risk actions require a second authorized principal and an immutable audit event. Client access is restricted to records explicitly owned or shared under the same tenant.

Pipeline contract

Every inbound request receives a correlation ID. Every command has an idempotency key. Domain changes and outbound messages use a transactional outbox. Workers lease tasks, retry with bounded exponential backoff, and move exhausted work to a dead-letter queue. Webhooks require signature verification, timestamp tolerance, replay protection, schema validation, and tenant resolution.

Transmission moves only through: draft → validated → awaiting approval → approved → queued → sent → acknowledged → accepted/rejected. Any identity, authority, environment, schema, signature, or reconciliation failure moves the item to quarantine.

Required production resources

- separate development, preview, test, and production environments
- managed relational database with point-in-time recovery
- KMS-backed secrets and encryption keys
- private encrypted object storage with malware scanning

- durable queue, scheduler, webhook receiver, and dead-letter queue
- centralized logs, metrics, traces, alerting, and audit export
- backups, restore testing, incident response, retention, legal hold, and vendor review

No external integration is considered active until credentials, scopes, callback URLs, signatures, sandbox tests, production approval, monitoring, and rollback have all been verified.

Execution policy

Decision classes

• Low risk: read, classify, draft, calculate, and prepare internal tasks.

• Controlled: import authorized transcripts, contact a client, generate a filing packet, or change workflow state. Requires authentication, tenant match, permissible purpose, and audit event.

• High risk: transmit, approve, submit to an agency, release money, change identity or authorization, export taxpayer data, or delete records. Requires explicit permission, MFA, separation of duties, idempotency, and authoritative confirmation.

Automations may prepare but may not silently approve or transmit. “Prepared,” “queued,” “sent,” “received,” “accepted,” and “resolved” are distinct states. Failed verification moves work to quarantine.

Andrea may explain, summarize, compare, calculate, cite, and draft. Andrea must state uncertainty, use current official sources for unstable rules, protect taxpayer data, and route material tax conclusions to a qualified human reviewer. Andrea does not impersonate the IRS, fabricate transcript data, provide unsupported refund dates, or promise code reversals.

Production deployment runbook

1. Confirm the approved legal-entity and DBA matrix.
2. Rotate every credential previously shared outside the secrets manager.
3. Create isolated development, preview, test, and production environments.
4. Configure database, identity, encrypted storage, queue, email, Stripe, and approved IRS integrations.
5. Run `scripts/runtime/preflight.mjs` and `npm run check`.
6. Apply database migrations with a verified backup and rollback plan.
7. Deploy preview and complete accessibility, security, RBAC, tenant-isolation, webhook, idempotency, and recovery tests.
8. Obtain compliance and owner approval.
9. Promote the same immutable artifact to production.
10. Validate `/api/health`, logs, traces, alerts, webhooks, callbacks, and a synthetic non-taxpayer workflow.
11. Record release evidence and monitor. Roll back on authentication, isolation, reconciliation, or acknowledgment failure.